Top 50 Most-Asked SQL Questions

SQL Interview Prep — 2026 Edition · Learn & Excel

The intel buyer's lifeline. Print, scan in 2 hours, walk into Friday's SQL interview. 50 questions across 5 sections: Joins & Subqueries (10), Window Functions (10), Indexes & Performance (10), Transactions & Isolation (10), Modern SQL Hot Topics (10).

This is the heart of the SQL interview pack: thirty questions that keep showing up across TCS, Infosys, Wipro, Accenture, Cognizant, GCC panels (JP Morgan India, Walmart Labs, Wells Fargo, Goldman, Target, AmEx) and product companies (Razorpay, Swiggy, Cred, Meesho, Zerodha, Flipkart, PhonePe). I've been in your shoes — these are the questions where candidates either nail it in 60 seconds or wander for five minutes and lose the round. Each page below gives you the 30-second answer first, then the layered explanation, then the edge case the interviewer is secretly waiting to hear. Read it like you'd read a friend's notes — fast, then slow on the bits you can't explain out loud.

Section A — Joins & Subqueries

Q1 · INNER vs LEFT vs RIGHT vs FULL OUTER JOIN

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Accenture, Wipro · Topic: Joins

The question (interviewer's verbatim phrasing):

> Can you walk me through the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN and FULL OUTER JOIN? Give me a real example where each one would return different rows.

The 30-second answer: INNER keeps only matched rows from both tables; LEFT keeps every row from the left table plus matches from the right (NULLs where no match); RIGHT is the mirror of LEFT; FULL OUTER keeps everything from both sides and pads with NULLs wherever a match is missing.

Step-by-step thought process: 1. Open with the matched-vs-unmatched framing — that's what the interviewer is testing. 2. Use a concrete example: `orders` and `payments`. INNER gives you only

orders that have a payment row. LEFT gives you all orders, including those without payments (useful for finding payment failures). RIGHT gives you all payments, including orphan payment rows. FULL OUTER gives you both sets of orphans in one go. 3. Mention that RIGHT JOIN is rarely used in production — most people flip the table order and use LEFT JOIN for readability. 4. Be ready for the follow-up: "How would you use LEFT JOIN to find orders without payments?" Answer: `LEFT JOIN payments p ON o.id = p.order_id WHERE p.order_id IS NULL` — this is the anti-join pattern.

Edge cases to mention: - If the join condition matches multiple rows on the right, INNER and LEFT both duplicate the left row — junior candidates forget this. - FULL OUTER JOIN is not supported in MySQL at all (still missing in 8.0 and 8.4) — you simulate it with `UNION ALL` of a LEFT JOIN and an anti-joined LEFT JOIN. - NULLs in the join column never match, even to other NULLs — that's standard SQL three-valued logic.

What NOT to say: - Do not say "LEFT JOIN includes all rows from both tables" — that's wrong, it only includes all rows from the left. - Do not draw Venn diagrams as your only explanation; interviewers want row-level reasoning, not pictures.

Common follow-up: "*If a candidate writes a LEFT JOIN but puts the right-table filter in the WHERE clause, what happens?*" — It silently converts to an INNER JOIN because the NULLs get filtered out. Put the filter in the ON clause instead.

```
-- Find orders that never got paid
SELECT o.id, o.user_id, o.amount
FROM orders o
LEFT JOIN payments p ON o.id = p.order_id
WHERE p.order_id IS NULL;
```

Q2 · Self-Join — Manager-Employee Pattern

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, TCS, Wells Fargo · Topic: Joins

The question (interviewer's verbatim phrasing):

> You have an employees table with `id`, `name` and `manager_id` where `manager_id` is a self-reference. Write a query to list every employee with their manager's name.

The 30-second answer: Join the table to itself with two aliases — one for the employee row, one for the manager row — matching `e.manager_id = m.id`, and use LEFT JOIN so the CEO (no manager) doesn't disappear.

Step-by-step thought process: 1. State the core idea: a self-join is just a join where both sides reference the same table with different aliases. 2. Write the LEFT JOIN — emphasise LEFT because the

top-level manager has a NULL `manager_id` and would otherwise vanish. 3. Mention common variations: finding pairs of employees with the same manager, or hierarchical levels (which leads into recursive CTEs). 4. Anticipate the follow-up about multi-level hierarchy — that requires a recursive CTE, not a single self-join.

Edge cases to mention: - Cycles: if `A` manages `B` and `B` somehow manages `A`, a recursive query loops forever — use a depth limit. - The CEO row will show `NULL` as manager name with LEFT JOIN — use `COALESCE(m.name, 'No manager')` to display cleanly. - If the same employee can have multiple managers (matrix orgs), one self-join row per manager is correct — explain the cardinality.

What NOT to say: - Do not use the same alias for both sides; the query will not parse and you'll look careless. - Do not skip the LEFT JOIN and then claim "the top manager is also in the result" — INNER JOIN drops them.

Common follow-up: *"Now show me each employee, their manager and their manager's manager — three levels."* — Either chain self-joins or, cleaner, use a recursive CTE.

```
SELECT e.name AS employee, COALESCE(m.name, 'No manager') AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

Q3 · Anti-Join and the NOT IN NULL Trap

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Goldman, JP Morgan · Topic: Joins

The question (interviewer's verbatim phrasing):

> I want all users who have never placed an order. Show me three different ways to write it and tell me which one is safest.

The 30-second answer: Three options — `LEFT JOIN ... WHERE order_id IS NULL`, `NOT EXISTS (...)`, and `NOT IN (...)`. The first two are safe; `NOT IN` is dangerous because if the subquery returns even a single NULL, the entire result becomes empty.

Step-by-step thought process: 1. Lead with the three forms — interviewer wants to know you've seen all of them. 2. Explain why `NOT IN` breaks: `x NOT IN (1, 2, NULL)` evaluates as `x != 1 AND x != 2 AND x != NULL`, and `x != NULL` is UNKNOWN, so the whole expression is UNKNOWN, which is not TRUE, so the row is filtered out. 3. State your preference: `NOT EXISTS` — it handles NULLs correctly and the optimizer treats it as an anti-semi-join. 4. Be ready to explain when `LEFT JOIN ... IS NULL` is preferred: when you also need columns from the left table efficiently.

Edge cases to mention: - If the column is declared `NOT NULL`, `NOT IN` is safe — but don't rely on schema you can't see. - `NOT EXISTS` short-circuits on the first match — usually faster on large right-side tables. - LEFT JOIN with IS NULL can be slower if the right table is huge and there are many matches that you then throw away.

What NOT to say: - Do not say "all three are equivalent." They are not equivalent in the presence of NULLs. - Do not assert one is always fastest — it depends on cardinality and indexes.

Common follow-up: "*What does the planner actually do with NOT EXISTS?*" — Postgres and modern SQL Server convert it to an anti-join operator, which is essentially a join that emits the left row only when no right match exists.

```
SELECT u.id, u.name
FROM users u
WHERE NOT EXISTS (
  SELECT 1 FROM orders o WHERE o.user_id = u.id
);
```

Q4 · EXISTS vs IN vs JOIN

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Walmart Labs, Cred · Topic: Joins

The question (interviewer's verbatim phrasing):

> When would you use EXISTS versus IN versus a JOIN? Are they always interchangeable?

The 30-second answer: They are semantically different — JOIN can multiply rows when the right side has duplicates, while EXISTS and IN return one row per left match. Pick EXISTS when you just need a check, JOIN when you need columns from the other table, IN when the list is small or comes from a constant set.

Step-by-step thought process: 1. Start with the cardinality point — that's the trap. A JOIN to `orders` from `users` where a user has 10 orders gives you 10 user rows; EXISTS gives you 1. 2. EXISTS is a semi-join: it stops scanning the inner table as soon as it finds the first match. 3. IN with a subquery is logically the same as EXISTS in most modern optimizers (Postgres, SQL Server, MySQL 8), but EXISTS reads more clearly when the inner query is correlated. 4. Anticipate: "Which is faster?" Answer: usually the same after the planner normalizes them — but EXISTS wins when the right table is huge and you have a covering index on the join column.

Edge cases to mention: - `IN` with a NULL in the list behaves differently from EXISTS — same NULL trap as anti-join. - If you JOIN and then use `DISTINCT` to dedupe, that's almost always slower than

rewriting as EXISTS. - In MySQL versions before 5.6, `IN (subquery)` was notoriously slow — modern versions fixed this.

What NOT to say: - Do not say "EXISTS is always faster than IN." Modern optimizers usually generate the same plan. - Do not blindly use JOIN when you only need a boolean check — you'll get duplicate rows.

Common follow-up: "*Rewrite this JOIN+DISTINCT query using EXISTS.*" — Practice this; it's a 30-second test of whether you actually understand the cardinality issue.

```
-- Users who have at least one successful payment
SELECT u.id, u.name
FROM users u
WHERE EXISTS (
  SELECT 1 FROM payments p
  WHERE p.user_id = u.id AND p.status = 'SUCCESS'
);
```

Q5 · Correlated Subquery — What and Why

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wells Fargo, Target · Topic: Subqueries

The question (interviewer's verbatim phrasing):

> What's a correlated subquery? Give me an example and tell me about its performance.

The 30-second answer: A correlated subquery references a column from the outer query, so it has to be re-evaluated for every outer row — conceptually it's a row-by-row loop. Modern optimizers often rewrite correlated EXISTS/IN into joins, but a correlated scalar subquery in the SELECT list usually stays as a per-row evaluation.

Step-by-step thought process: 1. Define it clearly: the inner query depends on a value from the outer query — that's the "correlation." 2. Show the classic example: "for each user, get the timestamp of their latest order." A correlated scalar subquery in the SELECT clause does this. 3. Explain the cost: if you have 1 million users and the inner subquery is not optimized, you do 1 million inner scans — O(N*M) in the worst case. 4. Offer the fix: rewrite as a window function (`ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC)`) or a join to a derived table that pre-aggregates.

Edge cases to mention: - A correlated subquery returning more than one row in a scalar context throws an error — guard with `LIMIT 1` or aggregation. - Postgres can sometimes decorrelate; SQL Server's plan will show a "Nested Loops" operator with an Apply. - Inside EXISTS, a correlated subquery is the idiomatic way and the planner handles it well.

What NOT to say: - Do not say "correlated subqueries are always slow" — for small outer sets or indexed inner queries they're fine. - Do not confuse "correlated" with "nested." All correlated subqueries are nested; not all nested subqueries are correlated.

Common follow-up: "*Rewrite this correlated scalar subquery using a window function.*" — Be fluent at this rewrite; it's a very common refactor in real systems.

```
-- Correlated: one inner scan per user
SELECT u.id,
       (SELECT MAX(created_at) FROM orders o WHERE o.user_id = u.id) AS last_order
FROM users u;
```

Q6 · Semi-Join

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Razorpay, Zerodha · Topic: Joins

The question (interviewer's verbatim phrasing):

> What's a semi-join? Why is EXISTS the natural way to write one?

The 30-second answer: A semi-join returns rows from the left table where at least one match exists in the right table, with no duplication and no right-side columns — essentially the "do any matching rows exist?" pattern, which EXISTS expresses directly.

Step-by-step thought process: 1. Contrast with INNER JOIN: INNER returns a row for every match (10 orders → 10 rows); semi-join returns one row per left match (10 orders → 1 row). 2. SQL has no `SEMI JOIN` keyword in most dialects — you express it via EXISTS, IN, or sometimes via JOIN+DISTINCT (worse). 3. Mention that the optimizer literally calls this operator a "semi-join" in EXPLAIN output — useful for reading query plans. 4. Anti-semi-join is the mirror — "no match exists" — expressed via NOT EXISTS.

Edge cases to mention: - DISTINCT after a JOIN works but is usually slower because the join materialises duplicates first, then dedupes. - Some warehouses (Snowflake, BigQuery) expose `LEFT SEMI JOIN` syntax directly — handy for ETL. - A semi-join through EXISTS short-circuits — the inner query stops at the first match per outer row.

What NOT to say: - Do not say "semi-join is a fancy term for INNER JOIN" — that misses the no-duplication point. - Do not write `JOIN ... GROUP BY` to dedupe when EXISTS does it cleaner.

Common follow-up: "*How would you write an anti-semi-join in standard SQL?*" — NOT EXISTS, or LEFT JOIN with IS NULL.

```
-- Semi-join via EXISTS
SELECT m.id, m.business_name
FROM merchants m
WHERE EXISTS (
  SELECT 1 FROM transactions t WHERE t.merchant_id = m.id AND t.amount > 100000
);
```

Q7 · LATERAL Join / CROSS APPLY

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Walmart Labs, Goldman · Topic: Joins

The question (interviewer's verbatim phrasing):

> Have you used LATERAL joins? When would you reach for one?

The 30-second answer: LATERAL (CROSS APPLY in SQL Server) lets the right side of a join reference columns from the left side — useful for per-row subqueries like "top 3 orders for each user" where a regular subquery cannot see the outer row.

Step-by-step thought process: 1. State the core capability: in a regular subquery in FROM, you cannot reference outer columns. LATERAL flips that — the right side is evaluated once per left row. 2. The textbook use case: top-N per group. `SELECT u.*, o.* FROM users u CROSS JOIN LATERAL (SELECT * FROM orders WHERE user_id = u.id ORDER BY created_at DESC LIMIT 3) o`. 3. Mention it's available in Postgres (`LATERAL`), SQL Server (`CROSS APPLY` and `OUTER APPLY`), and Oracle (12c+). MySQL added it in 8.0.14. 4. Contrast with window functions: for top-N, `ROW_NUMBER()` works for fixed N; LATERAL is cleaner when the inner query is complex or N varies.

Edge cases to mention: - `OUTER APPLY` / `LEFT JOIN LATERAL` keeps left rows even when the inner query returns zero rows. - Performance: planner typically uses a nested loop — fine for small left side, painful for huge ones. - The inner LIMIT is the killer feature — window functions cannot directly limit per partition without an outer filter.

What NOT to say: - Do not claim it's "just a subquery" — the correlation in FROM is the whole point. - Do not avoid it out of fear; it's standard SQL and shows seniority.

Common follow-up: *"Could you have written this with ROW_NUMBER instead?"* — Yes for simple top-N, but LATERAL is cleaner when the inner query has multiple joins or aggregates.

```
SELECT u.id, o.id AS order_id, o.amount
FROM users u
CROSS JOIN LATERAL (
  SELECT * FROM orders WHERE user_id = u.id ORDER BY created_at DESC LIMIT 3
) o;
```

Q8 · CROSS JOIN — Intentional vs Accidental

Tags: Difficulty: Easy · Asked at: TCS, Cognizant, Accenture, Infosys · Topic: Joins

The question (interviewer's verbatim phrasing):

> What's a CROSS JOIN and when would you actually want one?

The 30-second answer: A CROSS JOIN produces the Cartesian product — every row of A combined with every row of B, so N*M rows. Intentional use: generating combinations (dates × stores, sizes × colours). Accidental use: forgetting the join condition, which blows up your result set.

Step-by-step thought process: 1. Define Cartesian product crisply — N × M rows, no condition. 2. Give a real intentional example: a reporting query that needs every (store, date) pair so missing days appear as zero — you CROSS JOIN a stores table with a calendar table, then LEFT JOIN sales. 3. Mention the accidental form: writing `FROM orders, payments` without a WHERE join condition is an implicit CROSS JOIN — a classic junior mistake that takes down production. 4. Anticipate the follow-up about explicit syntax — always prefer `CROSS JOIN` keyword over comma-separated tables; it documents intent.

Edge cases to mention: - CROSS JOIN with a WHERE clause is equivalent to an INNER JOIN — but the explicit syntax is clearer. - 1000 × 1000 = 1 million rows. 100k × 100k = 10 billion. Cardinality goes up fast. - Some optimizers warn or refuse without explicit `CROSS JOIN` keyword — treat that as a feature, not annoyance.

What NOT to say: - Do not say "CROSS JOIN is always a mistake." Calendar joins, combination generation, and density tables all need it. - Do not use the comma syntax in modern code; it hides intent.

Common follow-up: "*How would you generate a row for every (date, store) pair in the last 30 days, even when there are no sales?*" — CROSS JOIN a date series with stores, then LEFT JOIN sales.

```
SELECT d.day, s.id, COALESCE(SUM(o.amount), 0) AS revenue
FROM generate_series(CURRENT_DATE - 29, CURRENT_DATE, '1 day') AS d(day)
CROSS JOIN stores s
LEFT JOIN orders o ON o.store_id = s.id AND o.order_date = d.day
GROUP BY d.day, s.id;
```

Q9 · Multi-Table Join Order

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Wells Fargo, AmEx · Topic: Joins

The question (interviewer's verbatim phrasing):

> Does the order in which I write joins in a query matter for performance?

The 30-second answer: For standard inner joins the optimizer is free to reorder them, so the order you write usually doesn't matter logically. But the join *shape* — star vs chain vs bushy — affects cost, and outer joins (LEFT/RIGHT/FULL) restrict reordering, so written order can matter there.

Step-by-step thought process: 1. Start with the cost-based optimizer point: Postgres, SQL Server, Oracle all reorder INNER joins using statistics. You write what reads well; the planner picks the order. 2. Outer joins are not freely reorderable because they change which rows survive. The planner respects the written nesting. 3. Mention the planner's join-order search limit: Postgres uses exhaustive dynamic programming until the join list exceeds `geqo_threshold` (default 12 items), at which point it switches to genetic optimization (GEQO). Separately, `join_collapse_limit` (default 8) controls how many explicit JOIN expressions get flattened into a single FROM list for reordering. 4. Real-world tip: when joining 10+ tables, break into CTEs or temp tables to constrain the planner and stabilise execution time.

Edge cases to mention: - `STRAIGHT_JOIN` in MySQL forces the written order — last-resort hint when the optimizer gets it wrong. - Statistics being stale is the most common reason a planner picks a bad join order — run `ANALYZE` after large data changes. - In Postgres, `join_collapse_limit = 1` disables reordering entirely — useful for debugging but not production.

What NOT to say: - Do not say "always join the smallest table first." The planner already knows table sizes; that advice is from a different era. - Do not claim outer-join order doesn't matter — it does, semantically.

Common follow-up: "*You have a 10-table query that's slow. How do you debug the join order?*" — Run EXPLAIN ANALYZE, check estimated vs actual row counts, refresh statistics, and consider breaking the query into stages.

Q10 · Subquery in SELECT vs FROM vs WHERE

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Swiggy, Target · Topic: Subqueries

The question (interviewer's verbatim phrasing):

> A subquery can sit in the SELECT list, in the FROM clause, or in the WHERE clause. What's the difference and when do you use which?

The 30-second answer: SELECT subquery returns one scalar value per row (correlated, often slow); FROM subquery (derived table) materialises a temporary result set you can join to; WHERE subquery is for filtering (EXISTS, IN, scalar comparison).

Step-by-step thought process: 1. Walk through each position with a concrete example. SELECT subquery: `SELECT u.id, (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.id) AS order_count FROM users u`. FROM subquery: `FROM (SELECT user_id, SUM(amount) AS total FROM orders GROUP BY user_id) o`. WHERE subquery: `WHERE user_id IN (SELECT id FROM users WHERE country = 'IN')`. 2. State the performance angle: SELECT subqueries are usually correlated and run once per row. FROM subqueries (derived tables / CTEs) are evaluated once. WHERE subqueries via EXISTS/IN are usually optimized into semi-joins. 3. Mention CTEs as the modern alternative to FROM subqueries — same execution but far more readable for multi-step logic. 4. Anticipate: "When would you choose a window function instead of a SELECT subquery?" — When you need the value per row and the inner query depends on the outer row, window functions are usually faster and clearer.

Edge cases to mention: - SELECT subqueries must return exactly one row and one column — otherwise runtime error. - A derived table in FROM needs an alias in standard SQL (Postgres enforces this strictly; MySQL is lenient). - Postgres materialises CTEs by default before version 12; from 12 onwards it inlines them like derived tables.

What NOT to say: - Do not say "they're all the same, just different syntax." The semantics and performance differ meaningfully. - Do not nest 5 levels of subqueries when a CTE chain would read cleanly.

Common follow-up: "*Rewrite this scalar subquery in SELECT as a LEFT JOIN to a derived table.*" — Practise this; it's the most common interview refactor.

Section B — Window Functions

Q11 · ROW_NUMBER vs RANK vs DENSE_RANK

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Walmart Labs, Cred · Topic: Windows

The question (interviewer's verbatim phrasing):

> What's the difference between ROW_NUMBER, RANK and DENSE_RANK? Give me a case where picking the wrong one breaks your query.

The 30-second answer: ROW_NUMBER gives a unique sequence with no ties (1, 2, 3, 4); RANK gives ties the same number but skips the next value (1, 1, 3, 4); DENSE_RANK gives ties the same number with no gaps (1, 1, 2, 3). Picking ROW_NUMBER for "top 3 by revenue" when there are ties silently drops valid tied rows.

Step-by-step thought process: 1. Open with the tie behaviour — that's the whole point of the question. 2. Use the canonical example: ordering students by marks. If two students tie at 95, ROW_NUMBER assigns them 1 and 2 arbitrarily; RANK gives both 1 and the next gets 3; DENSE_RANK gives both 1 and the next gets 2. 3. State the practical rule: ROW_NUMBER for pagination and "give me one row per group"; DENSE_RANK for "top N including ties"; RANK when the gap is semantically meaningful (Olympic medals). 4. Anticipate the trap: `WHERE rn = 1` with ROW_NUMBER for top-per-group works, but `WHERE rnk <= 3` with RANK can return more than 3 rows because of skipped numbers — usually you want DENSE_RANK here.

Edge cases to mention: - ROW_NUMBER with a non-unique ORDER BY is non-deterministic — tied rows can come back in any order. Add a tiebreaker column. - RANK skipping numbers can confuse downstream code that assumes sequential ranks. - All three respect PARTITION BY — counting resets per partition.

What NOT to say: - Do not say "they're basically the same." Tied data exposes the difference instantly. - Do not use ROW_NUMBER for "top 3 by score" interview questions when the interviewer probably wants ties included.

Common follow-up: "*Top 3 highest-paid employees per department, including ties.*" — DENSE_RANK with `rnk <= 3` is the canonical answer.

```
SELECT *, DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
FROM employees
QUALIFY rnk <= 3;  -- Snowflake/BigQuery; else wrap in subquery
```

Q12 · PARTITION BY vs GROUP BY

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, AmEx · Topic: Windows

The question (interviewer's verbatim phrasing):

> What's the conceptual difference between PARTITION BY in a window function and GROUP BY?

The 30-second answer: GROUP BY collapses rows — you get one row per group. PARTITION BY divides rows into groups for the window function to operate on, but every input row is preserved in the output.

Step-by-step thought process: 1. Lead with row preservation — that's the headline difference. 2. Concrete example: `SELECT user_id, amount, AVG(amount) OVER (PARTITION BY user_id) FROM orders`. You see every order plus that user's average alongside. With GROUP BY you'd see one row per user with the average only. 3. Mention they can coexist: you can GROUP BY one column and use window functions over the grouped result for cross-group comparison (running totals over weekly aggregates, for example). 4. Anticipate: "When would you use PARTITION BY without ORDER BY?" Answer: for partition-wide aggregates like average, count, sum where you don't need ordering.

Edge cases to mention: - A window function with no PARTITION BY treats the whole result set as a single partition — useful for "share of total" calculations. - Window functions execute after WHERE/GROUP BY/HAVING but before ORDER BY/LIMIT — this ordering matters for the filter-window trap (Q18). - Performance-wise, PARTITION BY usually requires a sort or hash — heavy on huge tables.

What NOT to say: - Do not say "PARTITION BY is the same as GROUP BY" — losing the row-preservation distinction is a red flag. - Do not GROUP BY when you wanted a window function — you'll lose row-level detail and have to re-join.

Common follow-up: "*Show me each order, the user's total spend, and the order's share of that total — in one query.*" — Window function with PARTITION BY plus arithmetic.

Q13 · Frame Clauses — ROWS vs RANGE

Tags: Difficulty: Hard · Asked at: Goldman, Razorpay, Walmart Labs, JP Morgan · Topic: Windows

The question (interviewer's verbatim phrasing):

> Walk me through frame clauses. What's the difference between ROWS BETWEEN and RANGE BETWEEN, and what does UNBOUNDED PRECEDING mean?

The 30-second answer: ROWS counts physical rows; RANGE groups rows with the same ORDER BY value into a logical band. UNBOUNDED PRECEDING means "from the start of the partition." Default frame for most aggregates is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which silently includes ties — the source of many bugs.

Step-by-step thought process: 1. Define the frame: a sliding window of rows over which the function aggregates, anchored at the current row. 2. Show ROWS vs RANGE with an example. Three rows with ORDER BY `day` where two rows share the same day: ROWS BETWEEN 1 PRECEDING AND CURRENT ROW gives the previous row and current; RANGE includes all rows tied with the current row's

day. 3. UNBOUNDED PRECEDING = from start. UNBOUNDED FOLLOWING = to end. CURRENT ROW = the current row. These combine to define the window. 4. The trap: the default frame for SUM/AVG with ORDER BY is `RANGE UNBOUNDED PRECEDING TO CURRENT ROW`, so duplicate ORDER BY values get summed together as one step — usually not what you want. Specify `ROWS` explicitly.

Edge cases to mention: - Without ORDER BY, the default frame is the entire partition — useful for "share of partition total." - With ORDER BY, the default frame becomes "start to current row" — running total semantics with the RANGE tie behaviour. - `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` is the standard "7-day moving sum" pattern.

What NOT to say: - Do not use RANGE for moving averages — ties will lump rows together and corrupt the average. - Do not omit the frame and assume ROWS — the default is RANGE-based in most engines.

Common follow-up: "*Write a 7-day moving average of daily transaction volume.*" — `AVG(amount) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`.

```
SELECT day, amount,
       SUM(amount) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS rolling_7d
FROM transactions;
```

Q14 · LAG and LEAD

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Zerodha, Cred · Topic: Windows

The question (interviewer's verbatim phrasing):

> Explain LAG and LEAD. Show me how you'd compute month-over-month revenue change.

The 30-second answer: LAG returns the value from a previous row in the partition; LEAD returns the value from a following row. Both accept an offset and a default value for when the row doesn't exist — crucial for the first/last row in the partition.

Step-by-step thought process: 1. Define the signature: `LAG(column, offset, default) OVER (PARTITION BY ... ORDER BY ...)`. Offset defaults to 1, default value defaults to NULL. 2. Show the MoM example: `LAG(revenue, 1, 0) OVER (ORDER BY month)` then subtract to get the change. 3. Mention LEAD is the mirror — useful for "time to next event" calculations like session timeouts or delivery latency. 4. Anticipate the gotcha: NULL default vs 0 default. If you subtract from NULL, you get NULL, not the first row's full value. Specify the default explicitly.

Edge cases to mention: - For the first row in a partition, LAG returns the default (NULL unless you specify). - Without PARTITION BY, LAG operates over the whole result set — useful for time-series with one entity. - LAG/LEAD don't accept the frame clause — they reference specific rows, not a range.

What NOT to say: - Do not write a self-join with `row_number - 1` to simulate LAG — slow and ugly. - Do not forget the default value when downstream code chokes on NULL.

Common follow-up: "*Now compute the percentage change, not absolute.*" — `(current - prev) / NULLIF(prev, 0)` — guard against division by zero.

```
SELECT month, revenue,
       revenue - LAG(revenue, 1, 0) OVER (ORDER BY month) AS mom_change
FROM monthly_revenue;
```

Q15 · FIRST_VALUE / LAST_VALUE Frame Trap

Tags: Difficulty: Hard · Asked at: Goldman, Walmart Labs, JP Morgan, AmEx · Topic: Windows

The question (interviewer's verbatim phrasing):

> If I write `LAST_VALUE(amount) OVER (PARTITION BY user_id ORDER BY created_at)`, what value do I actually get?

The 30-second answer: You get the amount of the current row, not the last row in the partition — because the default frame with ORDER BY is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, so "last" means last-seen-so-far. To get the true partition last, specify `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

Step-by-step thought process: 1. State the trap up front — interviewers love this question because most candidates get it wrong. 2. Explain the why: window frames default to "start to current row" with ORDER BY, so LAST_VALUE returns the current row's value, not the partition's last value. 3. Give the fix: explicit frame `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`. Now you actually get the last value. 4. Mention FIRST_VALUE works correctly with the default frame because the first row of "start to current row" is always the partition's first row.

Edge cases to mention: - An alternative for "last value" is `FIRST_VALUE` with a reversed ORDER BY — sometimes cleaner. - NTH_VALUE has the same frame dependency — explicit frame essential. - This trap doesn't exist in some warehouse dialects that default differently — but trust the standard behaviour.

What NOT to say: - Do not say "LAST_VALUE returns the partition's last row" without the frame caveat — that's the wrong answer. - Do not omit the frame "to keep the query short" — correctness first.

Common follow-up: "*Get the first and last transaction amount per user, in one query.*" — Two window expressions with explicit frames.

```
SELECT user_id, created_at, amount,
       FIRST_VALUE(amount) OVER (PARTITION BY user_id ORDER BY created_at) AS first_amt,
       LAST_VALUE(amount) OVER (PARTITION BY user_id ORDER BY created_at
                                ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_
FROM transactions;
```

Q16 · Running Totals

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy · Topic: Windows

The question (interviewer's verbatim phrasing):

> Compute the running total of daily revenue.

The 30-second answer: `SUM(revenue) OVER (ORDER BY day)` — with ORDER BY, the default frame accumulates from the start of the partition to the current row, which is the running total semantics.

Step-by-step thought process: 1. State the one-liner immediately — it's a short answer question, don't over-engineer. 2. Mention that without PARTITION BY, the running total runs across the whole result. Add `PARTITION BY user_id` for per-user running totals. 3. Make the default frame explicit if asked — `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` for safety, since RANGE with ties can lump days together (Q13). 4. Anticipate the variant: "running total of the last 7 days" — change frame to `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`.

Edge cases to mention: - Duplicate dates with RANGE-based default frame will share the same running total — usually wrong; use ROWS. - Missing days are silently skipped — if you need contiguous days, CROSS JOIN with a date series first. - Performance: a single window sort is usually faster than self-joining the table to itself.

What NOT to say: - Do not write a correlated subquery `(SELECT SUM(...) FROM t2 WHERE t2.day <= t1.day)` — that's O(N^2) and the wrong answer in 2026. - Do not omit ORDER BY — without it, SUM returns the partition total on every row, not a running total.

Common follow-up: "*What if I want running total to reset every month?*" — `PARTITION BY DATE_TRUNC('month', day) ORDER BY day`.

```
SELECT day, revenue,
       SUM(revenue) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS runnin
FROM daily_revenue;
```

Q17 · NTILE

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, AmEx, Wells Fargo · Topic: Windows

The question (interviewer's verbatim phrasing):

> What does NTILE do? Will it always give you equal-sized buckets?

The 30-second answer: NTILE(n) divides the partition into n approximately-equal buckets and tags each row with its bucket number. Buckets are equal-sized when the row count divides evenly; otherwise, the first few buckets each get one extra row.

Step-by-step thought process: 1. Define the function: `NTILE(4) OVER (ORDER BY amount)` puts every row into one of 4 buckets (quartiles). 2. Show the unequal case: 10 rows into 4 buckets — buckets 1 and 2 get 3 rows each, buckets 3 and 4 get 2 rows each. The "spillover" rows go to the front. 3. Common use cases: assigning customers to deciles by spend, splitting test/control groups, A/B bucketing. 4. Anticipate the trap: NTILE does not group by value — two rows with the same ORDER BY value can land in different buckets. Use PERCENT_RANK or manual bucketing for value-based bucketing.

Edge cases to mention: - With PARTITION BY, NTILE buckets reset per partition. - For percentile-style bucketing where ties must share a bucket, use `WIDTH_BUCKET` or arithmetic on PERCENT_RANK instead. - NTILE on a small partition (fewer rows than buckets) leaves trailing buckets empty.

What NOT to say: - Do not say "NTILE gives exactly equal buckets" — wrong for non-divisible row counts. - Do not confuse NTILE buckets with percentiles — they're row-based, not value-based.

Common follow-up: "*How would you assign users to deciles by lifetime spend?*" — `NTILE(10) OVER (ORDER BY total_spend)`.

Q18 · The Filter-Window Trap

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Flipkart · Topic: Windows

The question (interviewer's verbatim phrasing):

> Why can't I write `WHERE ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC) = 1` ? And what's the right way?

The 30-second answer: WHERE runs before window functions are evaluated, so the window function isn't visible at WHERE time. Wrap the query in a subquery or CTE and filter on the window output in the outer query — or use QUALIFY (Snowflake, BigQuery, Teradata).

Step-by-step thought process: 1. Explain the logical execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → window functions → DISTINCT → ORDER BY → LIMIT. Window functions evaluate after WHERE. 2. Show the two fixes. First, the portable one — wrap in a CTE: `WITH ranked AS (SELECT *, ROW_NUMBER() OVER (...) AS rn FROM t) SELECT * FROM ranked WHERE rn = 1`. Second, the warehouse-only one: `SELECT * FROM t QUALIFY ROW_NUMBER() OVER (...) = 1`. 3. Mention QUALIFY is supported in Snowflake, BigQuery, Teradata, Databricks SQL — not in Postgres, MySQL, or SQL Server yet. 4. Anticipate the follow-up about HAVING: HAVING also runs before window functions, so it has the same restriction.

Edge cases to mention: - Subquery vs CTE: in modern Postgres, both inline the same way. In older Postgres (<12), CTEs were optimization fences. - QUALIFY is cleaner but locks you to specific dialects — call this out in code reviews. - You can use a window function inside an ORDER BY clause directly — that one position works because ORDER BY runs after window evaluation.

What NOT to say: - Do not propose "just add it to WHERE" without acknowledging the order problem — interviewers want to hear you understand why it fails. - Do not assume QUALIFY exists everywhere; ask which warehouse the team uses before writing it.

Common follow-up: "*What's the difference between filtering with WHERE vs HAVING vs QUALIFY?*" — WHERE filters rows before grouping, HAVING filters after grouping but before windows, QUALIFY filters after windows.

```
-- Latest order per user (portable form)
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC) AS rn
  FROM orders
)
SELECT * FROM ranked WHERE rn = 1;
```

Q19 · PERCENT_RANK and CUME_DIST

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs, Target · Topic: Windows

The question (interviewer's verbatim phrasing):

> What's the difference between PERCENT_RANK and CUME_DIST?

The 30-second answer: Both return a value between 0 and 1 reflecting position in the partition. PERCENT_RANK is `(rank - 1) / (total_rows - 1)` — the relative rank, starts at 0.

CUME_DIST is `count_of_rows_with_value_<=_current / total_rows` — the cumulative distribution, never returns 0.

Step-by-step thought process: 1. State the formulas — interviewers want to see you actually know the maths, not just hand-wave. 2. PERCENT_RANK is "what fraction of rows are strictly below me?" — useful for percentile-style rankings. 3. CUME_DIST is "what fraction of rows are at or below me?" — used in statistical analysis and CDF computations. 4. Example: 4 rows with values 10, 20, 20, 30. For value 20: PERCENT_RANK = (2-1)/(4-1) = 0.333; CUME_DIST = 3/4 = 0.75.

Edge cases to mention: - Ties get the same value for both functions — they're not based on row position alone. - PERCENT_RANK on a single-row partition is 0 (no other rows to be ranked against), not NULL. - CUME_DIST is monotonically non-decreasing as you walk the ORDER BY — useful sanity check.

What NOT to say: - Do not confuse with NTILE — NTILE gives bucket numbers, not fractions. - Do not say they're "basically the same" — the inclusion-of-current-row difference matters in statistics.

Common follow-up: "*When would you use PERCENT_RANK in a real analytics query?*" — Computing percentile-rank salaries, exam scores, or merchant performance bands.

Q20 · Top-N-Per-Group via Window Functions

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, PhonePe · Topic: Windows

The question (interviewer's verbatim phrasing):

> Get the top 3 highest-amount orders for each customer. Why is a window function the canonical answer?

The 30-second answer: Number rows within each customer partition using `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC)`, then keep where the row number is ≤ 3. A pre-window GROUP BY can't do this because GROUP BY collapses rows; you'd need an awkward correlated subquery instead.

Step-by-step thought process: 1. State the pattern. Number rows per partition, then filter. Three steps. 2. Pick the right ranking function: ROW_NUMBER if you want exactly 3 even with ties, DENSE_RANK if "top 3 including ties" is what they mean. 3. Show the CTE wrapper (filter trap from Q18) — `WITH ranked AS (...) SELECT * FROM ranked WHERE rn <= 3`. 4. Mention the alternative — LATERAL join (Q7) — which is cleaner when the inner logic is complex but slower when N is small and the left side is huge.

Edge cases to mention: - Ties in amount with ROW_NUMBER will arbitrarily pick a winner — add a tiebreaker like `id` in the ORDER BY. - For "top 3 including ties" use DENSE_RANK ≤ 3 — can return

more than 3 rows per partition. - Performance: window functions sort once per partition — much faster than correlated subqueries.

What NOT to say: - Do not use a correlated subquery `(SELECT COUNT(*) FROM o2 WHERE o2.customer_id = o1.customer_id AND o2.amount > o1.amount) < 3` — works but slow. - Do not LIMIT in the inner query — LIMIT is per-query, not per-partition. Window function or LATERAL is the right tool.

Common follow-up: "*What if I want top 3 per customer, but only for the last 90 days?*" — Add `WHERE created_at >= CURRENT_DATE - 90` in the CTE before windowing.

```
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC, id) AS rn
  FROM orders
)
SELECT * FROM ranked WHERE rn <= 3;
```

Section C — Indexes & Performance

Q21 · B-tree Index Internals

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Walmart Labs · Topic: Indexes

The question (interviewer's verbatim phrasing):

> Explain how a B-tree index works internally. Why is lookup O(log n)?

The 30-second answer: A B-tree stores keys sorted in a balanced tree of pages — each internal page holds key ranges that point to child pages; leaf pages hold the actual keys plus row pointers (or, in clustered indexes, the row itself). Lookup walks from root to leaf, one page read per level, and tree depth grows logarithmically with row count.

Step-by-step thought process: 1. Sketch the structure: root page → internal pages → leaf pages, all balanced and sorted. 2. Explain the height. With page fan-out of ~100-500, a 1 billion row index is only 4-5 levels deep — so a lookup is 4-5 page reads, hence O(log n). 3. Mention what's at the leaf: in Postgres, a heap row pointer (CTID); in MySQL InnoDB, the primary key index leaf contains the actual row, and secondary index leaves contain the primary key (the "clustered index" model). 4. Anticipate the follow-up: range scans walk leaves left-to-right via sibling pointers — O(log n + k) where k is the matched range size.

Edge cases to mention: - B-tree is best for equality and range; hash indexes win on pure equality (Q25). - Insertions can cause page splits — high write rates fragment the index over time, hence periodic

REINDEX/OPTIMIZE. - Leaf pages typically hold dozens to hundreds of keys per 8KB page; depth stays small even for huge tables.

What NOT to say: - Do not say "B-tree is a binary tree." It's not — it's a high-fan-out balanced tree (the B is for "balanced," not "binary"). - Do not claim O(1) lookup — that's hash territory, not B-tree.

Common follow-up: "*Why does a B-tree work well for ORDER BY queries?*" — Because the leaves are stored in sorted order, the engine can stream them out without an extra sort.

Q22 · Composite Index Column Order

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Swiggy, Flipkart · Topic: Indexes

The question (interviewer's verbatim phrasing):

> If I create an index on `(user_id, created_at, status)`, can the database use it for a query that filters only on `created_at`?

The 30-second answer: No — composite indexes follow the leftmost-prefix rule. The index can serve queries that filter on `user_id`, or `user_id + created_at`, or all three; it cannot efficiently serve a query that filters only on `created_at` or only on `status`, because the leading column is required.

Step-by-step thought process: 1. Explain the physical layout: a composite index sorts rows by the first column, then by the second within ties, then the third. Without the first-column filter, the index is just a giant unsorted list with respect to the columns you care about. 2. State the leftmost-prefix rule clearly. Filter on `(user_id)`, `(user_id, created_at)`, or `(user_id, created_at, status)` — yes. Filter on `(created_at)` alone or `(status)` alone — no. 3. Mention the "index skip scan" feature (Oracle, MySQL 8) — it can sometimes use a non-leading column on low-cardinality leading columns, but it's slow and usually a sign you need a different index. 4. Anticipate: "What's the right column order?" Equality columns first, then range columns, then often-ORDER-BY columns last.

Edge cases to mention: - Postgres can sometimes do a "bitmap index scan" combining two separate single-column indexes — but it's almost always slower than a properly designed composite index. - For ORDER BY to use the index, the ORDER BY columns must match the index column order (and direction, unless the engine supports backward scans). - An index on `(a, b)` covers queries that filter on `a` and order by `b` — that's a key composite-index use case.

What NOT to say: - Do not say "the index covers any column listed in it." That's the trap. - Do not create one composite index per query — index maintenance cost adds up (Q29).

Common follow-up: "*I have queries that filter on `(user_id, created_at)` and queries that filter only on `created_at`. What do I do?*" — Either create both `(user_id, created_at)` and

`(created_at)` indexes, or reverse the composite order if the `created_at` queries are more frequent.

Q23 · Covering Index and Index-Only Scans

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Walmart Labs, JP Morgan · Topic: Indexes

The question (interviewer's verbatim phrasing):

> What's a covering index? What's the INCLUDE clause in Postgres or SQL Server?

The 30-second answer: A covering index includes all columns needed by the query — both the filter/sort columns and the SELECT-list columns — so the engine can answer the query entirely from the index without touching the table. INCLUDE adds non-key columns to the index leaves so they're available without being part of the sort key.

Step-by-step thought process: 1. Define index-only scan. EXPLAIN will literally say "Index Only Scan" in Postgres or "Index Seek with no Key Lookup" in SQL Server — the engine never reads the heap. 2. Show the value of INCLUDE. `CREATE INDEX ... ON orders (user_id) INCLUDE (amount, status)` — the index is still sorted by `user_id`, but the leaf pages also store `amount` and `status` so `SELECT amount, status FROM orders WHERE user_id = ?` is fully covered. 3. Why not just put everything in the key? Wider keys mean fewer entries per page, deeper trees, more write cost. INCLUDE keeps the key narrow but the leaf rich. 4. Anticipate: "When is index-only scan not actually free?" — In Postgres, visibility map must be up to date; otherwise the engine still has to verify rows in the heap. Run VACUUM regularly.

Edge cases to mention: - MySQL doesn't have INCLUDE syntax; you add extra columns to the index key — wider but works. - Covering indexes on wide tables can be much larger than the indexed columns alone — measure storage. - Postgres index-only scan can degrade if VACUUM hasn't updated the visibility map after heavy writes.

What NOT to say: - Do not say "covering index means SELECT *" — that's an anti-pattern; only cover what you actually need. - Do not assume covering always wins — the extra write/storage cost can hurt write-heavy tables.

Common follow-up: "*How do you tell if you got an index-only scan?*" — EXPLAIN ANALYZE will say "Index Only Scan" plus "Heap Fetches: 0" (Postgres) or no Key Lookup operator (SQL Server).

Q24 · When the Index Isn't Used

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, AmEx · Topic: Indexes

The question (interviewer's verbatim phrasing):

> I have an index on `email` but the query `WHERE LOWER(email) = 'x@y.com'` is doing a full table scan. Why?

The 30-second answer: Wrapping the column in a function (LOWER) prevents the engine from using the index, because the index is sorted by `email`, not by `LOWER(email)`. Fix it by storing emails normalized, by creating a functional index on `LOWER(email)`, or by ensuring the comparison is direct.

Step-by-step thought process: 1. State the principle: any operation that transforms the indexed column on the left side of the comparison (function, arithmetic, cast) disables the index. 2. List common offenders. `LOWER(email) = ?`, `email || '_' = ?`, `CAST(created_at AS DATE) = ?`, `created_at + INTERVAL '1 day' = ?`. Also implicit casts — comparing a `VARCHAR` column to an integer literal in MySQL casts every row. 3. Give fixes for each. Functional index in Postgres: `CREATE INDEX ON users (LOWER(email))`. For dates, rewrite `WHERE created_at >= '2026-01-01' AND created_at < '2026-01-02'` instead of `WHERE DATE(created_at) = '2026-01-01'`. 4. Other reasons indexes get skipped: low cardinality where a full scan is cheaper, `LIKE '%foo'` where the leading wildcard prevents B-tree use, OR conditions that the planner can't decompose, very small tables where index lookup overhead exceeds full scan cost.

Edge cases to mention: - LIKE 'foo%' can use a B-tree index (anchored prefix); LIKE '%foo%' cannot — use trigram or full-text indexes. - Implicit casts often happen with `WHERE phone_number = 9876543210` against a VARCHAR column — quote the literal. - The planner makes cost-based decisions — sometimes it's right to skip the index. Always check EXPLAIN with actual data sizes.

What NOT to say: - Do not say "the index is broken" or "the optimizer is wrong" without checking — usually it's a wrap, cast, or wildcard issue. - Do not force the index with hints as your first move; understand why it's skipped first.

Common follow-up: *"Show me how you'd debug an index-not-being-used issue end to end."* — EXPLAIN ANALYZE, check column transformations, check statistics with ANALYZE, check selectivity.

Q25 · Hash Index vs B-tree

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs, Cred · Topic: Indexes

The question (interviewer's verbatim phrasing):

> When would you choose a hash index over a B-tree?

The 30-second answer: Hash indexes give O(1) lookup for equality only — they cannot do ranges, ORDER BY, or prefix matches. So pick hash when you exclusively do equality lookups on a column and the table is huge enough that the constant-factor improvement matters; pick B-tree for everything else.

Step-by-step thought process: 1. Explain the data structures briefly. Hash index = hash table on disk: hash the key, get the bucket, find the row. B-tree = sorted balanced tree. 2. State the equality vs range distinction. Hash wins on equality alone. B-tree handles equality, range (`>` , `<` , `BETWEEN`), prefix LIKE, ORDER BY, all from the same structure. 3. Practical reality. In Postgres, hash indexes were unsafe before version 10 (no WAL logging); now they're production-ready but rarely chosen because B-tree is almost as fast on equality and far more flexible. In MySQL InnoDB, hash indexes exist as the "adaptive hash index" built automatically. 4. Anticipate: "Why don't we always use hash for primary keys?" — Because primary keys are often used in range scans and ORDER BY (pagination), where hash is useless.

Edge cases to mention: - Hash collisions degrade lookup; high-cardinality keys help. - Hash indexes don't support multi-column indexes meaningfully (you'd hash a tuple but lose the leftmost-prefix benefit). - In-memory hash tables (Redis-style) are different from disk-based hash indexes — don't confuse the two.

What NOT to say: - Do not say "hash is always faster than B-tree." For low-cardinality range queries B-tree wins; for tiny tables both are wasted. - Do not use hash for a column you'll later need to range-query — you'll have to rebuild.

Common follow-up: "*What is the InnoDB adaptive hash index?*" — InnoDB watches access patterns on B-tree pages; when it sees the same pages hit repeatedly, it builds an in-memory hash overlay automatically.

Q26 · Index Scan vs Index Seek vs Table Scan

Tags: Difficulty: Medium · Asked at: TCS, Wells Fargo, AmEx, Target · Topic: Indexes

The question (interviewer's verbatim phrasing):

> In EXPLAIN output, what's the difference between an index scan, an index seek, and a table scan?

The 30-second answer: Index seek navigates the B-tree to a specific key range — best case. Index scan reads the whole index sequentially — used when many rows match or no useful key range exists. Table scan (or sequential scan in Postgres) reads every heap row — used when no index helps or the table is small.

Step-by-step thought process: 1. Define each operator precisely. Seek = targeted descent into the tree. Scan = full traversal of the index leaves. Table/sequential scan = read heap pages directly. 2. State the cost ordering: seek (fastest, few pages) → index scan (medium, all index pages) → table scan (slowest

for selective queries, but fastest for full-table aggregates). 3. Mention the SQL Server terminology specifically — interviewers from MS-stack shops use these exact terms. Postgres EXPLAIN uses "Index Scan," "Index Only Scan," "Bitmap Index Scan," "Seq Scan." 4. Anticipate: "When is a table scan actually the right plan?" — When the query touches a large fraction of the table (say >10-20% of rows), the random I/O of index lookups exceeds the cost of sequential reads. Planner makes this call from statistics.

Edge cases to mention: - Index scan can be "Index Only" if all needed columns are in the index — never touches the heap. - Bitmap index scan combines multiple indexes by ORing/ANDing bitmaps — useful when no single composite index fits. - A "Seek + Key Lookup" plan (SQL Server) means the index seek found rows but had to fetch additional columns from the table — fix with covering index.

What NOT to say: - Do not say "table scans are always bad." For small tables or large fractions of the table, they're the right choice. - Do not assume "Index Scan" means seek — in Postgres EXPLAIN, "Index Scan" can mean full index traversal.

Common follow-up: "*Walk me through reading a Postgres EXPLAIN ANALYZE plan.*" — Lead with rows/loops, compare estimated vs actual, identify the most expensive operator.

Q27 · Reading EXPLAIN ANALYZE

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Goldman, Cred · Topic: Performance

The question (interviewer's verbatim phrasing):

> Walk me through how you'd read an EXPLAIN ANALYZE output. What does "cost" mean?

The 30-second answer: EXPLAIN shows the planner's predicted plan with estimated costs; EXPLAIN ANALYZE actually runs the query and adds real timing and row counts. "Cost" is an arbitrary internal unit — Postgres roughly calibrates 1.0 to a single sequential page read. The real signal is the gap between estimated and actual rows.

Step-by-step thought process: 1. Distinguish EXPLAIN (cheap, planning only) vs EXPLAIN ANALYZE (executes the query — beware of side effects on UPDATE/DELETE; wrap in BEGIN/ROLLBACK). 2. Explain the cost numbers. Cost is `startup_cost..total_cost` — startup is what's done before the first row is returned. The number itself is not seconds; it's a relative internal unit. 3. The most useful diagnostic: actual rows vs estimated rows. A 1000x mismatch means stats are stale (run ANALYZE) or the planner has bad correlation estimates — both lead to bad join orders. 4. Look for: full table scans on big tables, nested loops with high outer-row counts, hash joins spilling to disk (`Buffers: temp written=...`), and the order of operations (innermost = run first).

Edge cases to mention: - Always read EXPLAIN bottom-up — the deepest nested operator runs first. - Multiply Actual Rows × Loops for true row count in nested loop inner sides. - BUFFERS option (`EXPLAIN (ANALYZE, BUFFERS)`) shows page hits — useful for spotting cache effectiveness.

What NOT to say: - Do not say "the cost is in milliseconds." It isn't. Actual time is in EXPLAIN ANALYZE output, not in the cost. - Do not optimize without reading actual numbers — guessing usually makes things worse.

Common follow-up: *"A query is slow only in production. How would you diagnose?"* — Capture EXPLAIN ANALYZE in prod (with BUFFERS), compare against staging stats, check for plan flips due to changed data distribution.

Q28 · Low-Cardinality Index and Partial Indexes

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Walmart Labs, AmEx · Topic: Indexes

The question (interviewer's verbatim phrasing):

> I have an index on `status` which has only two values, ACTIVE and INACTIVE. The query planner ignores it. Why? And when would a partial index help?

The 30-second answer: Low-cardinality indexes get skipped because the planner estimates a large fraction of the table matches — at that selectivity, a sequential scan beats jumping through the index. A partial index `WHERE status = 'ACTIVE'` indexes only the rare rows, making it small and worth using.

Step-by-step thought process: 1. Explain selectivity. The planner estimates how many rows match. If matches are >5-10% of the table, the random I/O cost of index lookups exceeds sequential scan; planner picks seq scan. 2. With only two distinct values, status is splitting the table 50/50 or similar — index lookups would re-read most of the table anyway. 3. Partial index: `CREATE INDEX ON orders (created_at) WHERE status = 'ACTIVE'` — only indexes ACTIVE rows. Smaller index, faster lookups, and queries filtering for ACTIVE can use it. 4. Mention this is heavily used in soft-delete patterns: index only `WHERE deleted_at IS NULL`. Saves storage and improves lookup speed.

Edge cases to mention: - Partial indexes need the WHERE condition to be present in the query — exact match required. - MySQL doesn't have partial indexes; you can simulate with generated columns + index. - For low-cardinality columns where most queries filter the rare value, partial is the right answer; if both values are queried equally, neither index helps much.

What NOT to say: - Do not say "always index every column." Bad indexes hurt writes and waste cache (Q29). - Do not blame the planner for "ignoring" a useless index — it's making the right call.

Common follow-up: "*What about a multi-column index starting with status?*" — Generally still bad because leading low-cardinality columns force the index scan to read many pages; lead with the high-cardinality column instead.

```
-- Partial index for the common "active" filter
CREATE INDEX idx_active_orders_created
  ON orders (created_at)
  WHERE status = 'ACTIVE';
```

Q29 · Index Maintenance Cost

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Goldman · Topic: Performance

The question (interviewer's verbatim phrasing):

> What's the downside of adding indexes? Why can't you just index every column?

The 30-second answer: Every INSERT, UPDATE on an indexed column, and DELETE has to update each affected index — write amplification. Indexes also consume disk and buffer cache memory that could be used for hotter data. So more indexes = slower writes, larger storage, and sometimes worse read cache hit rates.

Step-by-step thought process: 1. State the write cost. An INSERT into a table with 5 indexes does 6 writes (1 heap + 5 indexes). UPDATE costs: if the updated column is indexed, the index entry has to move; if not, the index is untouched (HOT updates in Postgres). 2. Mention buffer cache competition. Indexes take memory; if indexes don't fit in RAM, you do random disk I/O for every lookup. Better to have a few well-targeted indexes that fit in cache. 3. Index bloat. Postgres MVCC keeps old tuples until VACUUM; indexes bloat with churn and need REINDEX. SQL Server / MySQL InnoDB have similar maintenance under heavy write loads. 4. Anticipate: "How do you decide which indexes to add?" — Profile actual slow queries with EXPLAIN, add the minimum number of composite indexes to cover the hottest patterns, monitor `pg_stat_user_indexes` for unused ones and drop them.

Edge cases to mention: - Bulk loads benefit from dropping indexes, loading, then recreating — much faster than maintaining indexes per row. - Indexes on UUID primary keys cause random insertion patterns and high page-split rates — consider time-ordered UUIDs (UUIDv7) or sequential IDs for write-heavy tables. - Unused indexes are pure overhead — Postgres `pg_stat_user_indexes` shows scan counts; drop anything with zero scans over a representative period.

What NOT to say: - Do not propose "index everything" — it's the most common junior mistake. - Do not ignore index maintenance jobs in monitoring; bloat sneaks up.

Common follow-up: "*How would you find unused indexes in production?*" — Postgres: `SELECT * FROM pg_stat_user_indexes WHERE idx_scan = 0`. SQL Server: `sys.dm_db_index_usage_stats`.

Q30 · Clustered vs Non-Clustered Index

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Wells Fargo, JP Morgan · Topic: Indexes

The question (interviewer's verbatim phrasing):

> Explain the difference between a clustered and a non-clustered index. How does MySQL InnoDB handle this versus Postgres?

The 30-second answer: A clustered index physically orders the table by the index key — the table itself is the index leaf. A non-clustered index is a separate structure pointing back to the row. MySQL InnoDB stores tables as a clustered B-tree on the primary key; secondary indexes store the PK, not a row pointer. Postgres has no clustered indexes — the heap is unordered (a "heap table"), and all indexes point to physical row addresses (CTIDs).

Step-by-step thought process: 1. Define both clearly. Clustered = table data IS the index leaf, sorted by index key. Non-clustered = separate B-tree pointing to row location. 2. MySQL InnoDB specifics. PRIMARY KEY is the clustered index — table is physically sorted by PK. Secondary index leaves contain the PK value, not a row pointer — so a secondary index lookup does two B-tree walks: secondary index → PK → row. This is why a short, monotonically increasing PK is critical for InnoDB. 3. Postgres specifics. All tables are heap-organized (unordered). All indexes are "secondary" and store the row's physical address (CTID). There's a `CLUSTER` command that physically reorders the heap by an index — but it's a one-time operation; new inserts don't maintain order. 4. SQL Server specifics: similar to InnoDB — every table can have one clustered index; non-clustered indexes store the clustering key.

Edge cases to mention: - InnoDB: choosing a random UUID as PK kills write performance because every insert lands at a random position in the clustered index, causing page splits. - Postgres CLUSTER takes an exclusive table lock — only viable for offline maintenance. - "Heap" in SQL Server means a table with no clustered index — generally avoided in production because non-clustered indexes then point to physical RIDs, which break on row movement.

What NOT to say: - Do not say "Postgres clustered indexes work like SQL Server's." They don't — Postgres lacks an always-on clustered storage model. - Do not assume the primary key is automatically clustered in every database — true for InnoDB and SQL Server default, but not for Postgres.

Common follow-up: "*Why does InnoDB recommend short integer primary keys?*" — Because the PK is embedded in every secondary index leaf; a long PK bloats every index.

Section D — Transactions & Isolation

Q31 · ACID Explained Practically

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay · Topic: Transactions

The question (interviewer's verbatim phrasing):

> Explain ACID. Don't recite the textbook — tell me what each letter actually means when you're writing payment code.

The 30-second answer: ACID = Atomicity (all-or-nothing), Consistency (DB ends in a valid state per its rules), Isolation (concurrent txns don't see each other's half-done work), Durability (once COMMIT returns, the data survives a crash).

Step-by-step thought process: 1. Atomicity — debit + credit in a fund transfer must both succeed or both roll back. If only the debit lands, money vanishes. 2. Consistency — DB-level invariants (FK, CHECK, NOT NULL, triggers) hold before and after the txn. App-level invariants (account balance >= 0) are your job, not the DB's. 3. Isolation — if two txns run concurrently, the outcome should look as if they ran one after the other. The isolation level controls how strict this is. 4. Durability — after COMMIT, even if the server is yanked from the rack, the change is on disk (WAL fsync'd).

Edge cases to mention: - Atomicity is per-transaction, not per-statement — a single INSERT that fails mid-way is auto-rolled back, but if you have 5 statements and statement 3 fails, you must explicitly ROLLBACK or the DB may leave 1 and 2 committed (depends on the driver / autocommit setting). - Durability has gradations — `synchronous_commit = off` in Postgres trades durability for throughput; a crash can lose the last few txns. - Consistency in CAP theorem is a different word — that's about replicas converging, not about DB invariants.

What NOT to say: - "Consistency means the data is correct" — too vague, the interviewer wants the invariant-preservation meaning. - "Isolation means transactions are independent" — they are not always; READ UNCOMMITTED makes them very dependent.

Common follow-up: *Which of A-C-I-D do you most often see relaxed for performance, and where?*

```
-- Atomicity in action: classic fund transfer
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE id = 101;
UPDATE accounts SET balance = balance + 5000 WHERE id = 202;
-- If either UPDATE fails or app crashes, ROLLBACK leaves both untouched
COMMIT;
```

Q32 · Read Uncommitted Isolation Level

Tags: Difficulty: Easy · Asked at: Cognizant, Accenture, Wells Fargo India · Topic: Isolation

The question (interviewer's verbatim phrasing):

> What does READ UNCOMMITTED actually allow? Have you ever used it in production?

The 30-second answer: READ UNCOMMITTED allows "dirty reads" — a txn can read rows another txn has modified but not yet committed. You almost never want this. The legitimate use is approximate analytics queries on hot tables where you'd rather see slightly-wrong-but-fast data than block.

Step-by-step thought process: 1. Define it: lowest isolation level. No read locks, no waiting on writers. 2. The anomaly it permits — dirty read. Reader sees value A, writer rolls back, value A never existed. 3. Real use case — `SELECT COUNT(*) FROM orders WITH (NOLOCK)` in SQL Server for a dashboard tile where being off by 200 rows is fine. 4. Postgres doesn't really support it — request READ UNCOMMITTED and you get READ COMMITTED, because Postgres's MVCC never reads uncommitted versions anyway.

Edge cases to mention: - SQL Server `NOLOCK` hint is the same thing applied per-query; widely (over)used in legacy reporting code. - Beyond dirty reads, READ UNCOMMITTED can return the same row twice or skip rows entirely if a page split happens mid-scan in SQL Server. - MySQL InnoDB supports it but most teams stay on REPEATABLE READ (the default).

What NOT to say: - "It's faster so use it everywhere." Performance gain is small and the correctness loss is large. - "Postgres supports all four levels equally" — it does syntactically, but READ UNCOMMITTED silently behaves as READ COMMITTED.

Common follow-up: *Why is dirty read dangerous in a financial report?*

```
-- SQL Server pattern often seen in legacy dashboards
SELECT COUNT(*) FROM orders WITH (NOLOCK) WHERE status = 'PLACED';
-- Same effect:
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT COUNT(*) FROM orders WHERE status = 'PLACED';
```

Q33 · Read Committed Isolation Level

Tags: Difficulty: Medium · Asked at: PhonePe, Flipkart, JP Morgan India · Topic: Isolation

The question (interviewer's verbatim phrasing):

> READ COMMITTED is the default in most databases. What does it prevent, and what does it still let through?

The 30-second answer: READ COMMITTED prevents dirty reads — you only see committed data. It still allows non-repeatable reads (same row read twice in one txn returns different values) and phantom reads (same range query returns different row counts).

Step-by-step thought process: 1. Default in Postgres, Oracle, SQL Server (MySQL is the odd one — defaults to REPEATABLE READ). 2. Mechanism — each statement sees a fresh snapshot in MVCC engines; in lock-based engines, short read locks released immediately. 3. What it does prevent: dirty read. 4. What still bites you: non-repeatable read. Run `SELECT balance FROM accounts WHERE id = 101` twice in one txn — value can change between reads because another txn committed in between. 5. Phantom reads also possible: `SELECT COUNT(*) FROM orders WHERE status = 'PLACED'` twice can return different counts.

Edge cases to mention: - In Postgres, READ COMMITTED takes a new snapshot per statement, not per transaction — this is what causes the non-repeatable behavior. - If you need stability within a txn, escalate to REPEATABLE READ or SNAPSHOT. - `SELECT ... FOR UPDATE` even in READ COMMITTED locks the row, so you can read-then-update without losing the update.

What NOT to say: - "READ COMMITTED prevents all anomalies." It only prevents one. - "It's slow because of locking." In MVCC engines (Postgres, Oracle), READ COMMITTED uses snapshots, not read locks.

Common follow-up: *Give an example where non-repeatable read causes a bug.*

```
-- Txn A reads twice, Txn B commits in between
-- Txn A:
BEGIN;
SELECT balance FROM accounts WHERE id = 101;  -- 5000
-- (Txn B updates to 8000 and commits)
SELECT balance FROM accounts WHERE id = 101;  -- 8000 — non-repeatable
COMMIT;
```

Q34 · Repeatable Read & InnoDB's Gap Lock

Tags: Difficulty: Medium · Asked at: Swiggy, Meesho, Walmart Labs · Topic: Isolation

The question (interviewer's verbatim phrasing):

> What does REPEATABLE READ add on top of READ COMMITTED? And is there anything special about MySQL's behaviour here?

The 30-second answer: REPEATABLE READ guarantees that if you read the same row twice in one txn, you get the same value (no non-repeatable reads). MySQL InnoDB goes further than the SQL standard — it also uses "gap locks" on range queries, which means InnoDB at REPEATABLE READ also prevents phantom reads (in most cases).

Step-by-step thought process: 1. The standard definition: REPEATABLE READ prevents dirty + non-repeatable reads, but phantom reads are still legal. 2. Postgres at REPEATABLE READ uses snapshot isolation — phantoms are also prevented in practice (any change after txn start is invisible). But you can get serialization errors on write-write conflicts. 3. MySQL InnoDB does it through gap locks on indexes — when you SELECT a range, it locks not just matching rows but the "gaps" between them, preventing inserts that would change the result set. 4. SQL Server at REPEATABLE READ holds shared locks until txn end — phantoms still possible.

Edge cases to mention: - InnoDB's gap locks are why you sometimes see deadlocks on inserts that look unrelated — two txns lock overlapping gaps. - Postgres's REPEATABLE READ is closer to SNAPSHOT isolation; can throw "could not serialize access due to concurrent update" forcing app retry. - Oracle doesn't expose REPEATABLE READ — it has READ COMMITTED and SERIALIZABLE (which is really snapshot).

What NOT to say: - "REPEATABLE READ always prevents phantoms" — only true on MySQL InnoDB and Postgres; SQL Server still allows them. - "It's just a stricter lock" — in MVCC engines it's about snapshots, not locks.

Common follow-up: *Why does MySQL default to REPEATABLE READ when most others use READ COMMITTED?*

```
-- MySQL InnoDB at REPEATABLE READ
BEGIN;
SELECT * FROM orders WHERE amount BETWEEN 100 AND 500;
-- Gap locks placed; another txn cannot INSERT a new row with amount = 200
-- until this txn commits
COMMIT;
```

Q35 · Serializable Isolation

Tags: Difficulty: Hard · Asked at: Goldman Sachs India, JP Morgan, Razorpay · Topic: Isolation

The question (interviewer's verbatim phrasing):

> What does SERIALIZABLE guarantee, and why don't we just always use it?

The 30-second answer: SERIALIZABLE guarantees the outcome is as if all txns ran one-after-another. It prevents all anomalies including phantoms and write skew. The cost — either heavy locking (SQL Server style) or frequent serialization-failure retries (Postgres SSI), which both kill throughput.

Step-by-step thought process: 1. Define the guarantee — serial-equivalent execution. The strongest level in the SQL standard. 2. Two implementation flavours: - **Lock-based** (SQL Server, DB2) — range locks, predicate locks; readers block writers and vice versa. - **MVCC + SSI (Serializable Snapshot Isolation)** — Postgres tracks read-write dependencies and aborts one txn if a cycle forms. Optimistic — no blocking, but retries needed. 3. The reason most prod code doesn't use it — throughput drops sharply under contention. 4. Where it does make sense — short, critical txns (financial settlement, ledger entries) where correctness trumps throughput.

Edge cases to mention: - Oracle's "SERIALIZABLE" is actually snapshot isolation — does not prevent write skew. Real serializable is not available in Oracle. - Postgres SERIALIZABLE will throw `could not serialize access` errors; app code must implement retry-on-40001. - Even at SERIALIZABLE, you still need to worry about replication lag if reads go to a replica.

What NOT to say: - "It's just slower" — the failure mode is different too (retries, not just latency). - "SERIALIZABLE means single-threaded" — it doesn't; concurrency is allowed, the *outcome* must be equivalent to some serial order.

Common follow-up: *What is write skew? Give an example.*

```
-- Postgres pattern with retry
DO $$
BEGIN
  FOR i IN 1..3 LOOP
    BEGIN
      SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
      -- ... txn body ...
      EXIT;
    EXCEPTION WHEN serialization_failure THEN
      -- retry
    END;
  END LOOP;
END $$;
```

Q36 · Dirty Read vs Non-Repeatable Read vs Phantom Read

Tags: Difficulty: Medium · Asked at: TCS, Cognizant, Target India · Topic: Isolation

The question (interviewer's verbatim phrasing):

> Walk me through dirty read, non-repeatable read, and phantom read with a concrete example of each.

The 30-second answer: Dirty read = you see uncommitted data. Non-repeatable read = you read the same row twice and get different values. Phantom read = you re-run the same range query and get a different number of rows. They are progressively more subtle.

Step-by-step thought process: 1. **Dirty read** — Txn A updates balance from 5000 to 8000 but hasn't committed. Txn B reads 8000, makes a decision, then Txn A rolls back. B used a value that never existed. 2. **Non-repeatable read** — Txn A reads order_amount = 500. Txn B updates it to 700 and commits. Txn A reads it again — 700. Same row, different value within one txn. 3. **Phantom read** — Txn A: `SELECT COUNT(*) FROM orders WHERE status = 'PLACED'` returns 50. Txn B inserts 3 new placed orders, commits. Txn A re-runs the same query — 53. 4. Map them to levels: READ UNCOMMITTED allows all three. READ COMMITTED kills dirty. REPEATABLE READ kills dirty + non-repeatable. SERIALIZABLE kills all three.

Edge cases to mention: - Phantom read is about row *count* or set membership, non-repeatable read is about row *value*. Subtle distinction interviewers love. - "Lost update" is a separate fourth anomaly not in the SQL standard list — two txns both read 100, both add 10, both write 110; one update is lost. - "Write skew" is a fifth anomaly that only SERIALIZABLE (real) prevents.

What NOT to say: - Mixing up phantom and non-repeatable read — common candidate slip. - "Dirty read is rare" — actually impossible at READ COMMITTED and above; not rare, just disallowed.

Common follow-up: *Which of these can happen at the default isolation level in your favourite DB?*

```
-- Phantom read scenario
-- Txn A:
BEGIN; SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT COUNT(*) FROM orders WHERE created_at::date = CURRENT_DATE;  -- 120
-- (Txn B inserts 5 new orders today, commits)
SELECT COUNT(*) FROM orders WHERE created_at::date = CURRENT_DATE;  -- 125 — phantom
COMMIT;
```

Q37 · Deadlocks — Detection & Retry

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Infosys · Topic: Transactions

The question (interviewer's verbatim phrasing):

> What is a deadlock, how does the database handle it, and what should your application do about it?

The 30-second answer: A deadlock is when two txns each hold a lock the other wants — neither can proceed. The DB detects the cycle and kills one (the "victim"), returning an error. Your app must catch this error and retry the txn.

Step-by-step thought process: 1. Classic example: Txn A locks row 1 and asks for row 2; Txn B locks row 2 and asks for row 1. Cycle. 2. Detection — most DBs run a "wait-for graph" check every few hundred ms; on cycle, pick the victim (usually the one that has done less work) and roll it back. 3. The error code — Postgres: `40P01`, MySQL: `1213`, SQL Server: `1205`. App layer catches this specifically and retries with backoff. 4. Prevention beats handling — always acquire locks in a consistent order across txns (e.g., always lock the smaller id first). Keep txns short.

Edge cases to mention: - A pure "lock timeout" is *not* a deadlock — it's a long wait that hit `innodb_lock_wait_timeout` or `lock_timeout`. Different error code. - Distributed deadlocks across shards or microservices aren't detected by any single DB — you must build timeout + retry yourself. - InnoDB's gap locks create deadlocks that surprise developers — two inserts into the same range can deadlock.

What NOT to say: - "Deadlocks mean my schema is broken" — not necessarily; high-concurrency workloads can deadlock even with good schema. - "Just retry forever" — bounded retries with exponential backoff. Otherwise, you mask a real contention bug.

Common follow-up: *How would you reduce deadlock frequency without changing application logic?*

```
-- Always lock in the same order: smaller id first
BEGIN;
SELECT * FROM accounts WHERE id IN (101, 202) ORDER BY id FOR UPDATE;
-- now safe to UPDATE both in any order
UPDATE accounts SET balance = balance - 500 WHERE id = 101;
UPDATE accounts SET balance = balance + 500 WHERE id = 202;
COMMIT;
```

Q38 · Lock Escalation

Tags: Difficulty: Hard · Asked at: AmEx India, Goldman Sachs, Wipro · Topic: Transactions

The question (interviewer's verbatim phrasing):

> Tell me about lock escalation in SQL Server or Oracle — when does a row lock become a page or table lock?

The 30-second answer: Lock escalation is when the database decides that maintaining many fine-grained locks (row-level) is more expensive than holding one coarse lock (page or table). It "escalates" — releases the small locks, takes a big one. SQL Server kicks in around 5000 locks per statement; Oracle largely avoids row-lock escalation.

Step-by-step thought process: 1. Why locks aren't free — each lock takes memory (~96 bytes in SQL Server). Holding millions is expensive. 2. SQL Server's rule of thumb — at ~5000 locks on a single object in one statement, escalate row locks to a table lock (no intermediate page lock by default for HoBT escalation). 3. Effect on concurrency — once you have a table-level X lock, no other writer can touch any row of that table. 4. How to avoid — keep txns short, batch DELETE/UPDATE into smaller chunks (e.g., 1000 rows at a time), use partition-level locking via `ALTER TABLE ... SET (LOCK_ESCALATION = AUTO)`.

Edge cases to mention: - Oracle does not escalate row locks to table locks — it uses row-level locking with a per-block transaction list, so this question is largely SQL Server / DB2-specific. - MySQL InnoDB doesn't escalate either; uses row-level locking on indexed columns. - Postgres uses row-level + table-level locks but does not "escalate" in the SQL Server sense.

What NOT to say: - "All DBs escalate locks" — Oracle and Postgres really don't, in this sense. - "Escalation is always bad" — sometimes a table lock for a big batch is intentional and faster.

Common follow-up: *How would you do a bulk DELETE of 10 million rows without causing lock escalation?*

```
-- SQL Server: chunked delete to avoid escalation
WHILE 1 = 1
BEGIN
  DELETE TOP (1000) FROM orders WHERE status = 'EXPIRED' AND created_at < DATEADD(year, -3, GETDAT
  IF @@ROWCOUNT = 0 BREAK;
  WAITFOR DELAY '00:00:00.100';
END
```

Q39 · MVCC — How Snapshot Isolation Works

Tags: Difficulty: Hard · Asked at: Zerodha, PhonePe, Flipkart, JP Morgan · Topic: Transactions

The question (interviewer's verbatim phrasing):

> Explain MVCC. How does Postgres or Oracle give you snapshot isolation without making readers block writers?

The 30-second answer: MVCC = each row update creates a new version with a transaction-id stamp. Readers see the version visible at their snapshot's start; writers create new versions. Readers never block writers, writers never block readers — they look at different physical versions of the same logical row.

Step-by-step thought process: 1. The core idea — a row is not a single value; it's a chain of versions, each tagged with `xmin` (creator txn) and `xmax` (deleter txn). At read time, the engine picks the version visible to your snapshot. 2. Postgres mechanics — versions stored inline in the same table page (HOT chains when possible). Old versions cleaned up by VACUUM. 3. Oracle mechanics — old versions stored in UNDO segments, not in the table itself. Readers reconstruct old image from UNDO. This is why "snapshot too old" errors happen when UNDO is too small. 4. MySQL InnoDB — old versions in undo log; similar to Oracle but different on-disk layout. 5. Trade-off — bloat. Postgres in particular suffers if VACUUM falls behind on a heavy-write table.

Edge cases to mention: - MVCC does not eliminate locks — writers still take row locks to prevent two updates colliding. It eliminates *read* locks. - Long-running queries on a busy table can hold back VACUUM, causing table bloat. Common Postgres prod issue. - The "snapshot too old" Oracle error and Postgres's `xmin horizon` are the same family of problem — old version evicted before the reader was done.

What NOT to say: - "MVCC means no locks anywhere" — wrong; write locks still exist. - "MVCC and snapshot isolation are the same thing" — close but not identical. MVCC is the implementation; snapshot isolation is the guarantee.

Common follow-up: *What is `VACUUM` doing in Postgres, and what happens if it's too slow?*

```
-- See row visibility in Postgres
SELECT xmin, xmax, * FROM orders WHERE id = 101;
-- xmin = txn id that created this version
-- xmax = txn id that deleted/updated it (0 if still current)
```

Q40 · Optimistic vs Pessimistic Locking

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Meesho, TCS · Topic: Transactions

The question (interviewer's verbatim phrasing):

> When do you use `SELECT FOR UPDATE` versus a version-column check? What's the trade-off?

The 30-second answer: Pessimistic (`SELECT FOR UPDATE`) — lock the row up front, others wait. Use when conflict is likely and the txn is short. Optimistic (version column) — read freely, check on write that nobody changed it; if they did, retry. Use when conflict is rare and you don't want readers blocking each other.

Step-by-step thought process: 1. Pessimistic — `SELECT * FROM accounts WHERE id = 101 FOR UPDATE` puts a row lock; other writers block until commit. Safe and simple, but blocks. 2. Optimistic — add a `version` (or `updated_at`) column. On read, capture it. On UPDATE, set `version = version + 1` with a `WHERE version = :captured` . If 0 rows updated, somebody beat you to it — retry or fail. 3. Pessimistic is right for inventory decrements (likely contention), payment debits. 4. Optimistic is right for user-profile edits (rare contention), where the cost of a blocked reader is worse than the cost of occasional retry. 5. There's also "advisory locks" (Postgres `pg_advisory_lock`) for cross-row coordination that doesn't fit either pattern.

Edge cases to mention: - `SELECT FOR UPDATE NOWAIT` — fail instantly instead of waiting. Useful in API hot paths. - `SELECT FOR UPDATE SKIP LOCKED` — skip locked rows entirely; great for job-queue tables. - Optimistic needs careful handling of partial updates — if you UPDATE 5 columns conditionally, the version bump must cover all of them.

What NOT to say: - "Optimistic is always faster" — under high contention, retries thrash and it ends up slower than pessimistic. - "Pessimistic locks everything" — only the rows you SELECT FOR UPDATE; other readers without the clause still see them (in MVCC engines).

Common follow-up: *How would you implement a job-queue table so two workers don't pick the same job?*

```
-- Optimistic pattern with version column
-- Read:
SELECT id, balance, version FROM accounts WHERE id = 101;
-- captured version = 7

-- Write:
UPDATE accounts
   SET balance = balance - 500, version = version + 1
 WHERE id = 101 AND version = 7;
-- If 0 rows affected, retry
```

Section E — Modern SQL Hot Topics

Q41 · Recursive CTE

Tags: Difficulty: Medium · Asked at: TCS, Walmart Labs, JP Morgan India, Flipkart · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> Write a query that returns all employees under a given manager — direct reports, their reports, all the way down.

The 30-second answer: Use a recursive CTE — anchor member picks the seed (the manager), recursive member joins back to the CTE itself, walking the tree. Add a depth column if you need to know how far down each employee is.

Step-by-step thought process: 1. Recursive CTE has two parts joined by `UNION ALL` — the anchor (base case) and the recursive (steps). 2. Anchor: pick the starting manager(s). 3. Recursive: join the table to the CTE on `manager_id = parent.id`, returning the next layer. 4. Termination is automatic when the recursive step returns zero rows. 5. Use `depth + 1` to cap runaway recursion; add `LIMIT` defensively when testing.

Edge cases to mention: - Cycles (employee A reports to B reports to A) cause infinite loops. Track visited ids in an array (`PATH` column) and exclude in the recursive step. Postgres has `CYCLE` clause; SQL Server / MySQL don't. - `UNION ALL` (cheap, preserves dupes) vs `UNION` (forces distinct, much slower). Almost always `UNION ALL`. - MySQL added recursive CTEs in 8.0; before that you needed stored procs or self-joins.

What NOT to say: - "I'd write a stored procedure with a loop" — recursive CTE is the standard answer interviewers want. - "Recursive CTEs are O(1)" — they're O(rows in result), and bad anchors can explode them.

Common follow-up: *How would you find the shortest path between two users in a friend-graph table?*

```
WITH RECURSIVE reports AS (
  -- Anchor: the seed manager
  SELECT id, name, manager_id, 0 AS depth
    FROM employees WHERE id = 42
  UNION ALL
  -- Recursive: next layer down
  SELECT e.id, e.name, e.manager_id, r.depth + 1
    FROM employees e
    JOIN reports r ON e.manager_id = r.id
   WHERE r.depth < 10  -- safety cap
)
SELECT * FROM reports ORDER BY depth, name;
```

Q42 · UPSERT — MERGE / ON CONFLICT / ON DUPLICATE KEY

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, PhonePe, Infosys · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> I have a `users` table with `email` as unique. New signup comes in — if the email exists, update last_seen; if not, insert. How do you write this in one statement?

The 30-second answer: Postgres: `INSERT ... ON CONFLICT (email) DO UPDATE`. MySQL: `INSERT ... ON DUPLICATE KEY UPDATE`. SQL Server: `MERGE` (or workaround). Each has gotchas — MySQL touches the auto-increment counter on conflict, SQL Server MERGE has historical concurrency bugs.

Step-by-step thought process: 1. Postgres ON CONFLICT — specify the conflict target (unique index column), then DO UPDATE or DO NOTHING. Access the rejected row via `EXCLUDED.col`. 2. MySQL ON DUPLICATE KEY — checks all unique constraints, not a specific one. Use `VALUES(col)` or `NEW.col` in MySQL 8.0.20+. 3. SQL Server MERGE — verbose, but flexible. Has known race conditions if the unique index isn't there or if WHEN MATCHED conditions skip rows. 4. Vendor-portable workaround — INSERT, catch unique-violation, UPDATE. Two round-trips but no surprises.

Edge cases to mention: - MySQL `ON DUPLICATE KEY UPDATE` increments AUTO_INCREMENT *even when no insert happens* — your id sequence will have gaps. - Postgres `ON CONFLICT` requires a unique constraint or unique index — error if the conflict target doesn't have one. - SQL Server MERGE has been criticized — Aaron Bertrand's "Use Caution with MERGE" piece is the classic reference. Many shops use INSERT + UPDATE instead. - Returning the affected row — Postgres supports `RETURNING`; vanilla MySQL still does not (as of 8.4). MariaDB added `RETURNING` for INSERT/DELETE in 10.5, but Oracle MySQL has no equivalent — fall back to `LAST_INSERT_ID()` or a follow-up SELECT.

What NOT to say: - "SELECT first, then INSERT or UPDATE" — race condition; two concurrent calls both SELECT no row and both INSERT. - "MERGE is the standard ANSI way so use it everywhere" — performance and concurrency vary wildly by vendor.

Common follow-up: *What happens if two clients UPSERT the same email at the same instant?*

```
-- Postgres
INSERT INTO users (email, last_seen) VALUES ('a@b.com', NOW())
ON CONFLICT (email) DO UPDATE SET last_seen = EXCLUDED.last_seen;

-- MySQL
INSERT INTO users (email, last_seen) VALUES ('a@b.com', NOW())
ON DUPLICATE KEY UPDATE last_seen = VALUES(last_seen);
```

Q43 · GROUPING SETS / ROLLUP / CUBE

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target India, AmEx · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> A report needs total orders by city, by state, and overall — all in one result set. Without UNION ALL spaghetti. How?

The 30-second answer: Use `GROUPING SETS` to specify exactly which sub-totals you want. `ROLLUP` gives you a hierarchical chain; `CUBE` gives you every possible combination. All three reduce to one query plan and one scan of the source.

Step-by-step thought process: 1. `GROUP BY GROUPING SETS ((city), (state), ())` — three sub-totals: per city, per state, grand total. 2. `GROUP BY ROLLUP (state, city)` — hierarchical: (state, city), (state), (). Use when there's a natural parent-child. 3. `GROUP BY CUBE (a, b)` — every combination: (a, b), (a), (b), (). Use rarely; explodes with more columns. 4. `GROUPING(col)` function tells you whether a NULL in the output is a real NULL or a "this column was not grouped at this level" placeholder.

Edge cases to mention: - NULL ambiguity — a `state = NULL` row in the result can mean "state was rolled up" or "the underlying data had NULL state". Use `GROUPING()` to distinguish. - MySQL supports only `WITH ROLLUP` — GROUPING SETS and CUBE are still not implemented (true through 8.4). - BigQuery and Snowflake support all three; very useful in analytics dashboards.

What NOT to say: - "I'd write three queries with UNION ALL." Works, but the interviewer is specifically testing if you know these features. - "ROLLUP and CUBE are the same." They're not — ROLLUP is hierarchical, CUBE is exhaustive.

Common follow-up: *How do you distinguish a real NULL state from a rolled-up NULL state in the output?*

```
SELECT state, city, COUNT(*) AS orders_count,
       GROUPING(state) AS is_state_rollup,
       GROUPING(city)  AS is_city_rollup
  FROM orders
 GROUP BY GROUPING SETS ((state, city), (state), ());
```

Q44 · JSON in SQL — JSONB Operators & When to Use

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Meesho, Zerodha · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> Postgres has a `jsonb` column. Tell me about the operators and when you'd use a JSON column versus proper relational columns.

The 30-second answer: Postgres: `->` returns json, `->>` returns text, `@>` checks containment, `?` checks key existence. MySQL: `JSON_EXTRACT(col, '$.path')` or `col->'$.path'`. Use JSON columns for semi-structured payloads with sparse / evolving fields — never as a substitute for normal modelling of frequently-queried fields.

Step-by-step thought process: 1. Postgres operators: - `->` get json field, returns jsonb: `data->'address'` - `->>` get text field: `data->>'name'` - `@>` contains: `data @> '{"status":"active"}'` - `?` key exists: `data ? 'phone'` 2. Index them with GIN: `CREATE INDEX ON users USING gin(data jsonb_path_ops);` 3. When to use JSON column — variable user metadata, third-party webhook payloads, A/B test config blobs, audit log "context" fields. 4. When NOT to — anything you JOIN, ORDER BY, or filter on regularly. Make it a real column. 5. MySQL — similar set with `JSON_EXTRACT`, `JSON_CONTAINS`, but indexing requires generated columns.

Edge cases to mention: - Postgres `jsonb` is binary-stored and indexable; `json` is text-stored and only good for storage. Almost always pick `jsonb`. - Updates rewrite the whole document — heavy update workloads on JSON columns cause bloat. - BigQuery `JSON` type and Snowflake `VARIANT` are similar but with schema-on-read; even better for analytics.

What NOT to say: - "JSON columns make the schema flexible so always use them." That's how you end up with a slow, un-typed mess. - "JSON in SQL is for storage only." It's queryable with predicate pushdown if you index correctly.

Common follow-up: *How would you index `data->>'status' = 'active'` for fast lookup?*

```
-- Postgres
SELECT id FROM users WHERE data @> '{"city":"Bengaluru"}';
SELECT id FROM users WHERE data->>'plan' = 'pro';

-- Index for @> queries
CREATE INDEX idx_users_data ON users USING gin (data jsonb_path_ops);

-- Index for ->> equality
CREATE INDEX idx_users_plan ON users ((data->>'plan'));
```

Q45 · CTE vs Subquery vs Temp Table

Tags: Difficulty: Medium · Asked at: Flipkart, JP Morgan, Wipro, Cognizant · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> When would you reach for a CTE versus a subquery versus a temp table?

The 30-second answer: Subquery — quick, inline, one-time use. CTE — same query but more readable and reusable within the statement. Temp table — when you need to materialize once and hit it many times across statements, or when the optimizer is making a bad plan and you want to force materialization.

Step-by-step thought process: 1. **Subquery** — fine for simple inline use; can sometimes be flattened by the optimizer. 2. **CTE** — readability win. In Postgres <12, CTEs were always materialized (acting as an optimization fence). From Postgres 12 onwards, CTEs are inlined by default — use `WITH ... AS MATERIALIZED` to force the old behaviour. 3. **Temp table** — `CREATE TEMP TABLE tmp AS SELECT ...`. Persists for the session, can be indexed, statistics gathered. Best when reused across multiple queries. 4. Reusability rule of thumb — if you reference the result once, subquery or CTE; many times, temp table.

Edge cases to mention: - Postgres 12 inlining means your old-style "use a CTE to force materialization" trick no longer works. Add `AS MATERIALIZED` to keep it. - SQL Server temp tables (`#tmp`) get statistics; table variables (`@tmp`) don't and often plan badly. - BigQuery / Snowflake — temp tables are often slower than CTEs because of storage roundtrip; prefer CTEs there.

What NOT to say: - "CTEs are always faster than subqueries" — depends on version and engine. They're a readability tool first. - "Temp tables are slow" — often the fastest option when you're hitting the result repeatedly.

Common follow-up: *If a CTE is being inlined and the plan is bad, how do you force materialization?*

```
-- Postgres 12+: force materialization
WITH active_users AS MATERIALIZED (
  SELECT id, plan FROM users WHERE is_active = TRUE
)
SELECT a.plan, COUNT(*)
  FROM active_users a JOIN orders o ON o.user_id = a.id
 GROUP BY a.plan;
```

Q46 · Materialized View

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target India, Goldman, Razorpay · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> What is a materialized view and how is it different from a regular view? What refresh strategies have you used?

The 30-second answer: A regular view is just a saved query — re-runs every time. A materialized view stores the result on disk and serves it directly, much like a cached table. You refresh it on a schedule or trigger. Trade-off: fast reads, stale data, refresh cost.

Step-by-step thought process: 1. Use case — expensive aggregations (daily revenue per merchant per city) that downstream dashboards hit every 30 seconds. 2. Refresh strategies: - **Full refresh** — `REFRESH MATERIALIZED VIEW`. Simple, locks readers in plain form. - **Concurrent refresh** (Postgres) — `REFRESH MATERIALIZED VIEW CONCURRENTLY` — needs a unique index on the view; readers continue but refresh is slower. - **Incremental / delta refresh** — Oracle "fast refresh", Snowflake dynamic tables, BigQuery materialized views. Only re-computes changed rows. Best but has restrictions on the SQL. 3. Trigger-based refresh on small lookup-style MVs is also common. 4. The eternal tension — freshness vs cost. Daily? Hourly? Every 5 min? Match to the business question's tolerance.

Edge cases to mention: - Postgres has no auto-refresh; you schedule it (cron, pg_cron, app-side). - Snowflake auto-refreshes but charges credits for the refresh — review billing. - Materialized views can become invalid if the underlying tables change schema; check application deployment workflow.

What NOT to say: - "Materialized views are always faster" — only on read. Refresh cost can be brutal. - "Just use a cache like Redis" — sometimes right, but materialized views keep the data near the SQL engine which simplifies joins.

Common follow-up: *How do you avoid blocking readers during refresh?*

```
-- Postgres
CREATE MATERIALIZED VIEW mv_daily_revenue AS
  SELECT date_trunc('day', created_at) AS day,
         merchant_id, SUM(amount) AS revenue
    FROM orders WHERE status = 'PAID'
   GROUP BY 1, 2;

-- Required for CONCURRENTLY
CREATE UNIQUE INDEX ON mv_daily_revenue (day, merchant_id);

REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_revenue;
```

Q47 · Table Partitioning — Range, List, Hash

Tags: Difficulty: Hard · Asked at: Flipkart, PhonePe, Walmart Labs, JP Morgan · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> When does partitioning help and when does it hurt? Range, list, or hash — how do you pick?

The 30-second answer: Partitioning helps when queries naturally restrict to one partition (partition pruning) or when you need to drop old data fast (drop a partition vs a billion-row DELETE). It hurts when queries span partitions — partition overhead with no payoff. Range for time-series, list for fixed categories, hash to spread load evenly.

Step-by-step thought process: 1. **Range** — by date or numeric range. `PARTITION BY RANGE (created_at)`. Classic time-series; old partitions get archived/dropped. 2. **List** — by an enumerated set. `PARTITION BY LIST (region) (...)`. Useful when data has a natural categorical split (region, tenant). 3. **Hash** — `PARTITION BY HASH (user_id) PARTITIONS 16`. For load distribution when no natural range/list exists. 4. Wins: partition pruning skips entire partitions; DROP PARTITION is instant; per-partition statistics. 5. Losses: cross-partition queries pay overhead; foreign keys to partitioned tables have restrictions in older Postgres; partition key must be in WHERE for pruning to kick in.

Edge cases to mention: - Postgres declarative partitioning since 10; better in 11+ (default partition, hash partitions, FK support). - MySQL partitioning has limits — partition key must be part of every unique key. - Don't over-partition — 10000 partitions is too many; plan metadata overhead becomes significant. - Sub-partitioning (range + hash) is sometimes used in very large warehouses but adds complexity.

What NOT to say: - "Partitioning makes everything faster." Only specific access patterns benefit. - "Always range-partition by date." Only if your queries actually filter by date.

Common follow-up: *Why might a partitioned table be slower than a non-partitioned one for some queries?*

```
-- Postgres range partitioning by month
CREATE TABLE orders (
  id BIGSERIAL, created_at TIMESTAMP NOT NULL, amount NUMERIC
) PARTITION BY RANGE (created_at);

CREATE TABLE orders_2026_05 PARTITION OF orders
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');
-- Pruning works: WHERE created_at >= '2026-05-15' scans only May partition
```

Q48 · COALESCE / NULLIF / IS DISTINCT FROM

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> Two columns. How do you check if they're different, treating NULLs as just another value to compare?

The 30-second answer: `a = b` returns NULL when either is NULL — never TRUE, never FALSE. Use `IS DISTINCT FROM` (Postgres / SQL Server 2022 / Snowflake) for NULL-safe inequality. Use COALESCE to substitute a default; use NULLIF to convert a sentinel value to NULL.

Step-by-step thought process: 1. **The NULL trap** — `WHERE old_value <> new_value` will silently exclude rows where either side is NULL. Probably not what you want for change detection. 2. **`IS DISTINCT FROM`** — NULL-safe inequality: NULL vs 5 is "distinct", NULL vs NULL is "not distinct". Postgres has it; MySQL uses `<=>` (NULL-safe equals); SQL Server 2022 finally added it. 3. **`COALESCE(a, b, c)`** — returns the first non-NULL. Useful for defaults and fallbacks. 4. **`NULLIF(a, b)`** — returns NULL if a = b, else a. Common trick: `NULLIF(divisor, 0)` to avoid divide-by-zero.

Edge cases to mention: - MySQL's `<=>` is its NULL-safe equals; `a <=> b` returns 1 if both NULL or both equal, 0 otherwise. - `IS NOT DISTINCT FROM` is the equality version. - `COALESCE` is ANSI standard; `ISNULL()` is SQL Server-specific and behaves slightly differently with type precedence. - `COALESCE` short-circuits — later expressions not evaluated if an earlier one is non-NULL.

What NOT to say: - "NULL = NULL is true." It's NULL, which is treated as false in WHERE clauses. - "Use `IFNULL` everywhere." MySQL/SQL Server have it but `COALESCE` is portable.

Common follow-up: *Write a query that finds rows where the user changed their email — including NULL-to-value and value-to-NULL.*

```
-- NULL-safe change detection
SELECT id FROM user_changes
 WHERE old_email IS DISTINCT FROM new_email;  -- Postgres / SQL Server 2022

-- MySQL equivalent
SELECT id FROM user_changes WHERE NOT (old_email <=> new_email);

-- Divide-by-zero guard
SELECT total / NULLIF(quantity, 0) AS unit_price FROM orders;
```

Q49 · CTE in DML — WITH ... UPDATE / DELETE / INSERT

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> In Postgres, you can use a CTE with INSERT, UPDATE, DELETE. Show me a useful pattern.

The 30-second answer: Postgres lets you stack data-modifying CTEs in one statement using `WITH ... INSERT`, `WITH ... UPDATE`, `WITH ... DELETE`. Combined with `RETURNING`, this gives you "do A and feed its output into B" — atomic, single round-trip.

Step-by-step thought process: 1. The canonical example — archive then delete: select rows to archive, insert into archive, delete from main, all in one statement. 2. `RETURNING` from the first CTE feeds the second. 3. Important quirk — sibling CTEs see a *snapshot* of the source tables; the order of effect is "as if all CTEs ran on the original state". A `DELETE` in CTE 1 isn't visible to a `SELECT` in CTE 2 against the same table. 4. Other engines vary — SQL Server allows `WITH` with `INSERT/UPDATE/DELETE` to define source data, but doesn't allow chaining multiple data-modifying CTEs. MySQL added basic support; Oracle's syntax is different.

Edge cases to mention: - Snapshot semantics — be careful when one CTE deletes and another counts rows; the count will be the pre-delete count. - Auditing / logging table inserts driven off updates — clean pattern using RETURNING. - Performance — one network round-trip, one transaction, one plan. Generally faster than two statements.

What NOT to say: - "WITH is read-only." Not in Postgres. - "All DBs support this pattern." Mostly Postgres-specific.

Common follow-up: *If a CTE deletes rows from `orders`, will a sibling CTE that SELECTs from `orders` see those deletions?*

```
-- Archive-and-delete pattern (Postgres)
WITH moved AS (
  DELETE FROM orders
   WHERE status = 'CANCELLED' AND created_at < CURRENT_DATE - INTERVAL '90 days'
  RETURNING *
)
INSERT INTO orders_archive
SELECT * FROM moved;

-- Update-and-audit pattern
WITH updated AS (
  UPDATE accounts SET balance = balance - 500 WHERE id = 101
  RETURNING id, balance
)
INSERT INTO audit_log (account_id, new_balance, ts)
SELECT id, balance, NOW() FROM updated;
```

Q50 · Window Function vs Aggregate in Same Query

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, JP Morgan India, Zerodha · Topic: Modern SQL

The question (interviewer's verbatim phrasing):

> Show me a query that returns each order's amount alongside the running total for that customer and the customer's grand total — all in one go.

The 30-second answer: Use a window function for the running total (it preserves the row) and either an aggregate window or a GROUP BY-based join for the grand total. The key idea: an aggregate function collapses rows, a window function keeps every row and computes alongside.

Step-by-step thought process: 1. The conceptual difference — `SUM(amount)` in `GROUP BY` returns one row per group. `SUM(amount) OVER (PARTITION BY customer_id)` returns the same value attached to every row of that group. 2. For a running total — `SUM(amount) OVER (PARTITION BY customer_id ORDER BY created_at)` — the `ORDER BY` inside `OVER` turns it from total to running. 3. For grand total per customer — `SUM(amount) OVER (PARTITION BY customer_id)` — no ORDER BY inside. 4. Mix them — three columns: amount (the row), running_total (with ORDER BY in OVER), customer_total (no ORDER BY).

Edge cases to mention: - `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` is the default frame when ORDER BY is present — that's why "running total" works. - Without ORDER BY in OVER, the frame is the whole partition, giving the partition total on every row. - Mixing `GROUP BY` and `OVER` is legal but tricky — the window function operates *after* GROUP BY. - Performance — one scan with window functions vs join-back to aggregate. Usually faster.

What NOT to say: - "Window functions are just a fancy GROUP BY." They preserve rows; GROUP BY collapses them. Different shape. - "Running totals need self-joins." Old pattern; window functions are the modern way.

Common follow-up: *What does the default window frame look like, and how would you change it to compute a 7-day moving average?*

```
SELECT
  id,
  customer_id,
  amount,
  SUM(amount) OVER (PARTITION BY customer_id ORDER BY created_at)
    AS running_total,
  SUM(amount) OVER (PARTITION BY customer_id)
    AS customer_grand_total,
  AVG(amount) OVER (PARTITION BY customer_id
                    ORDER BY created_at
                    RANGE BETWEEN INTERVAL '6 days' PRECEDING
                              AND CURRENT ROW)
    AS rolling_7d_avg
FROM orders
ORDER BY customer_id, created_at;
```